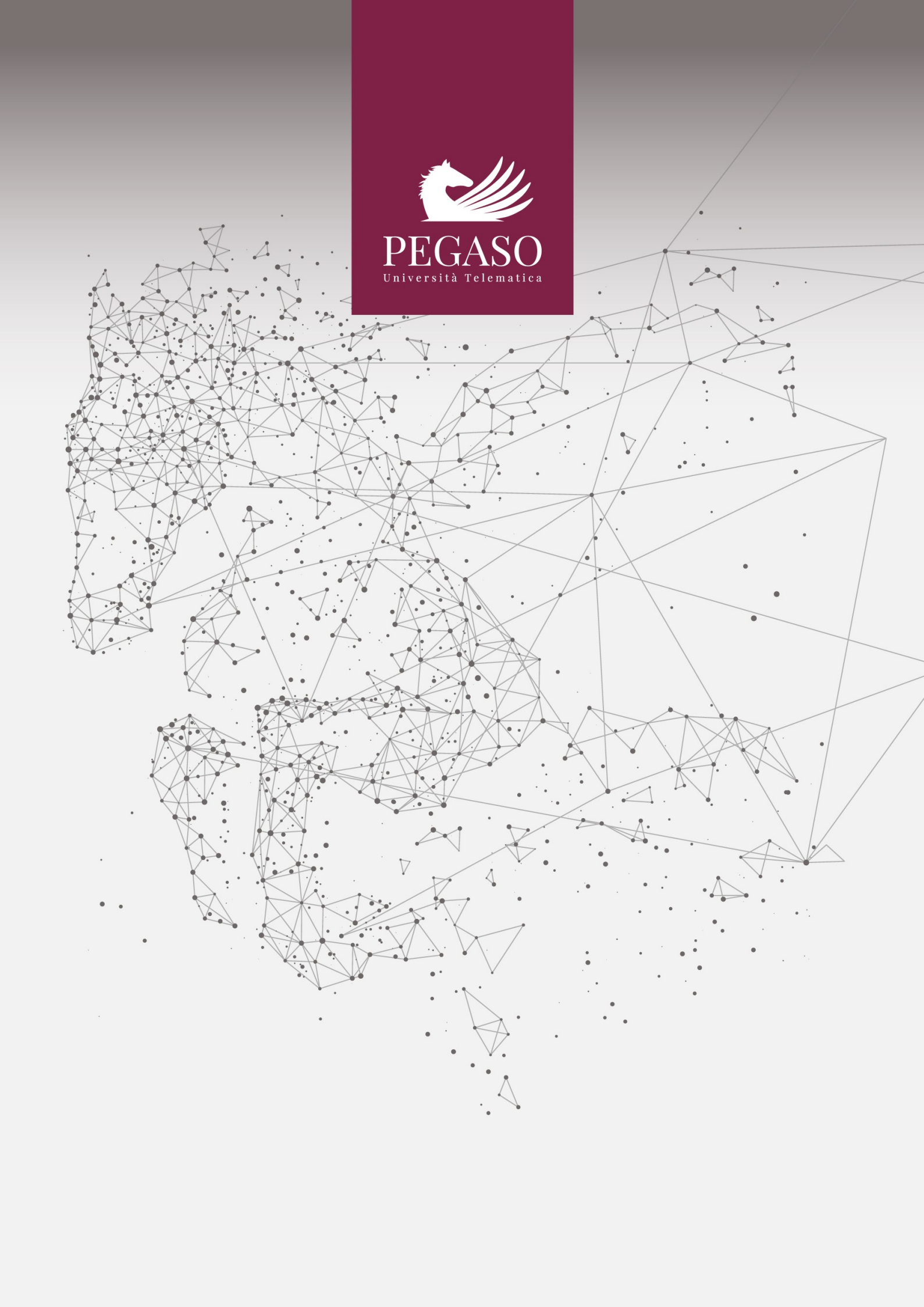

PEGASO
Università Telematica

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. OPERAZIONI DI BASE 3
2. PASSAGGIO DI PARAMETRI 7
3. SUPPORTO AD APPLICAZIONI BASATE SU RPC 9
BIBLIOGRAFIA 12

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Operazioni di base

Un gran numero di sistemi distribuiti si basa sullo scambio esplicito di messaggi tra processi. Tuttavia, le operazioni di *send* e *receive* non nascondono in alcun modo la comunicazione, che è fondamentale per ottenere la trasparenza dell'accesso nei sistemi distribuiti. Questo problema è stato riconosciuto per molto tempo, ma fino al 1980 poco è stato fatto per affrontarlo. Negli anni '80, tuttavia, i ricercatori hanno introdotto un metodo completamente nuovo di gestire la comunicazione. Sebbene il concetto sia piuttosto semplice (una volta concepito), le sue implicazioni sono spesso sottili. Questa lezione analizzerà il concetto, la sua applicazione, i suoi punti di forza e i suoi punti deboli.

In sostanza, la proposta era quella di consentire ai programmi di accedere alle procedure su altre macchine. Quando un processo sul computer **A** chiama una procedura sul computer **B**, il processo chiamante su **A** viene sospeso e la procedura viene eseguita su **B**. Attraverso l'inserimento di parametri, le informazioni possono essere trasmesse dal chiamante al destinatario della chiamata e restituite nel risultato della procedura. Nessuna trasmissione di messaggio è visibile al programmatore. Questo metodo viene definito chiamata di procedura remota (Remote Procedure Call o RPC in breve).

Mentre il concetto di base sembra semplice ed elegante, ci sono sottili difetti. A causa del fatto che le procedure chiamate e quelle chiamanti vengono eseguite su computer separati, e quindi eseguite in spazi di indirizzi separati, si possono avere complicazioni.

Inoltre, devono essere scambiati parametri e risultati tra le macchine, il che può essere complicato se esse non sono perfettamente identiche. In definitiva, una o entrambe le macchine potrebbero andare incontro a dei fallimenti, con ogni potenziale guasto che si traduce in complicazioni diverse a seconda del contesto. Tuttavia, la maggior parte di questi problemi è gestibile e RPC è una tecnica ampiamente utilizzata che è alla base di molti sistemi distribuiti.

L'obiettivo di RPC è fare in modo che una chiamata di procedura remota appaia il più locale possibile. In altre parole, vogliamo che la procedura di chiamata di RPC sia trasparente. Il chiamante in sostanza non deve essere consapevole del fatto che la procedura chiamata è in esecuzione su un computer diverso o viceversa. Si supponga che un programma abbia accesso a un database che gli consente di aggiungere dati a un elenco archiviato prima di restituire un puntatore all'elenco modificato. L'operazione è resa accessibile al programma tramite un'aggiunta di routine:

```
newlist = append(data, dbList)
```

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

In un sistema convenzionale (a processore singolo), il linker estrae la funzione *append* da una libreria e la inserisce nel programma oggetto. In teoria, potrebbe essere una breve procedura che potrebbe essere implementata con alcune operazioni sui file per l'accesso al database.

Anche se *append* esegue solo alcune operazioni di base sui file, viene chiamato nel modo convenzionale impilando i suoi parametri. Il programmatore non è a conoscenza dei dettagli di implementazione di *append*, che è come quanto previsto che sia.

RPC raggiunge la sua trasparenza in modo simile. Quando *append* è effettivamente una procedura remota, al client chiamante viene presentata una versione di *append* differente, detta client stub. Allo stesso modo dell'originale, è anche essa chiamata usando la sequenza di chiamata standard. A differenza dell'originale, tuttavia, questa versione non esegue un'operazione di *append*. Come illustrato nella Figura 1, incapsula i parametri all'interno di un messaggio e richiede che questo messaggio venga inviato al server.

Dopo aver chiamato *send*, il client chiama la funzione *receive*, quindi bloccandosi fino all'arrivo della risposta.

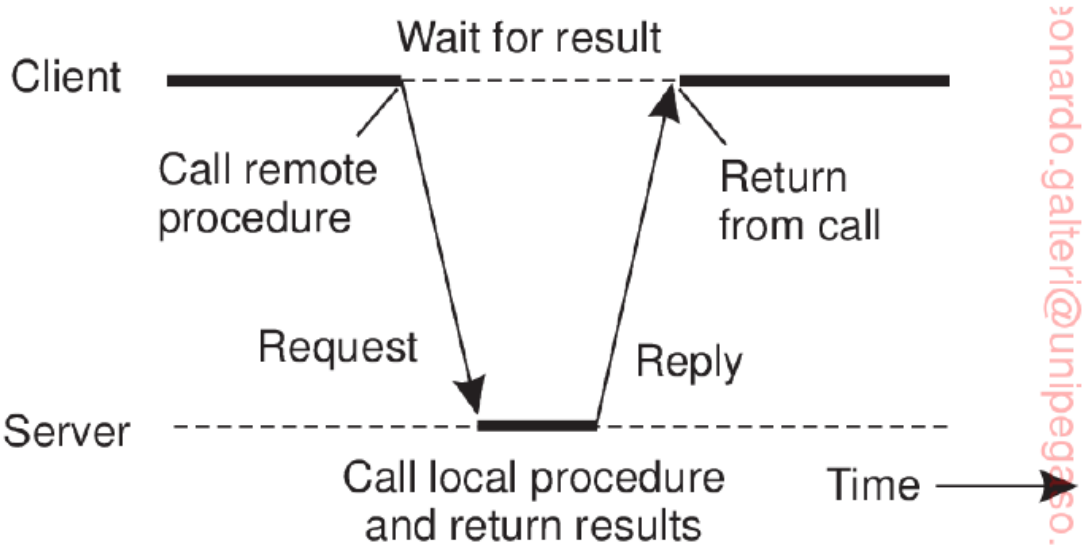


Figura 1 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Quando un messaggio raggiunge il server, il suo sistema operativo lo inoltra a un server stub. Un server stub è la controparte lato server di un client stub; si tratta di un pezzo di codice che trasforma le richieste di rete in chiamate di procedura locale. Il server stub avrà in genere chiamato una procedura *receive* e rimarrà bloccato in attesa di messaggi in arrivo. Il server stub decomprime i parametri del messaggio e quindi chiama la routine del server come al solito. Dal punto di vista del server, sembra che il

PEGASO Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

client lo stia chiamando direttamente; i parametri e l'indirizzo di ritorno sono sullo stack a cui appartengono e nulla sembra fuori dall'ordinario. Il server completa l'attività e restituisce il risultato al chiamante (in questo caso, il server stub) nel modo consueto.

Quando il server stub riprende il controllo dopo una chiamata riuscita, incapsula il risultato in un messaggio e chiama l'operazione *send* per restituirlo al client. Il server stub in genere chiama nuovamente *receive* per attendere la successiva richiesta in arrivo.

Quando il messaggio contenente il risultato arriva al computer del client, il sistema operativo lo passa al client stub tramite l'operazione *receive* chiamata in precedenza e il processo client viene quindi sbloccato.

Il client stub esamina il messaggio, decomprime il risultato, lo copia nel chiamante e quindi restituisce normalmente. Quando il chiamante riprende il controllo dopo la chiamata da aggiungere, sa solo che i dati sono stati aggiunti a un elenco. Non è a conoscenza del fatto che il lavoro è stato eseguito in remoto da un altro computer.

Per quanto riguarda il client, i servizi remoti sono accessibili tramite chiamate di procedura standard (cioè locali) e non tramite chiamate specifiche *send* e *receive*. Tutti i dettagli del passaggio dei messaggi sono nascosti all'interno delle due procedure di libreria, proprio come i dettagli delle chiamate di sistema sono nascosti all'interno delle librerie convenzionali.

In breve, una chiamata di procedura remota è costituita dai passaggi successivi:

1. In genere, la procedura client chiama il client stub in modo normale.
2. Il client stub costruisce un messaggio e comunica con il sistema operativo locale.
3. Il messaggio viene inviato dal sistema operativo del client al sistema operativo remoto.
4. Il sistema operativo remoto invia il messaggio al server stub.
5. Il server stub decomprime i parametri e quindi chiama il server.
6. Il server esegue l'operazione e restituisce il risultato allo stub.
7. Il server stub crittografa il risultato e chiama il suo sistema operativo locale.
8. Il messaggio viene inviato dal sistema operativo del server al sistema operativo del client.
9. Il sistema operativo del client invia il messaggio al client stub.
10. Lo stub decomprime e restituisce il risultato al client.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

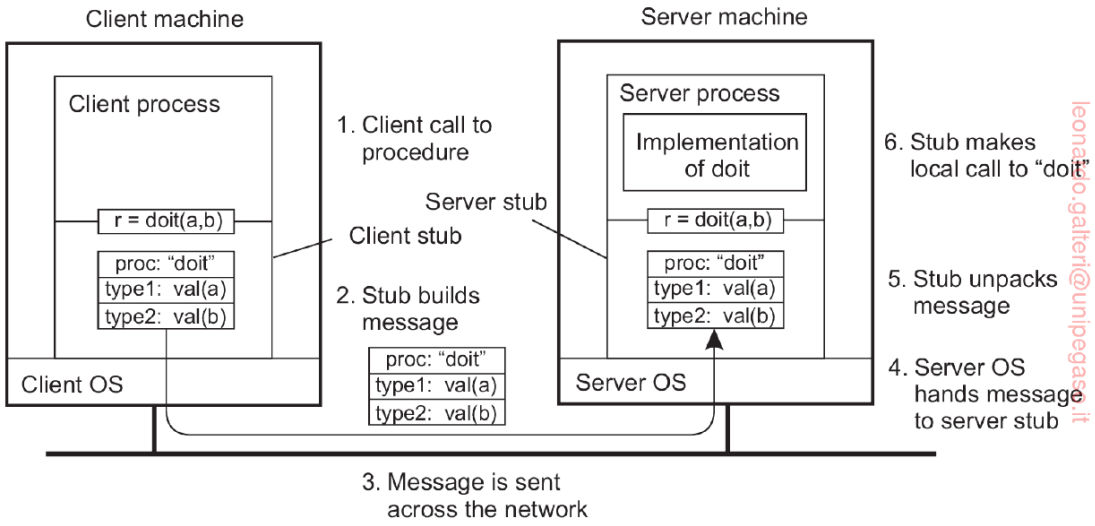


Figura 2 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

La Figura 2 illustra i passaggi iniziali per una procedura astratta a due parametri *doit(a,b)*, supponendo che il parametro **a** sia del tipo *type1* e il parametro **b** sia del tipo *type2*. Tutti questi passaggi hanno l'effetto netto di convertire la chiamata locale della procedura client al client stub in una chiamata locale alla routine server, senza che né il client né il server siano a conoscenza dei passaggi intermedi o dell'esistenza della rete.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Passaggio di parametri

Il client stub è responsabile della creazione di pacchetti dei parametri in un messaggio e dell'invio al server stub. Anche se sembra un'operazione semplice, non è così semplice come sembra inizialmente.

Il marshalling dei parametri si riferisce al processo di inclusione dei parametri in un messaggio. Tornando alla nostra operazione di *append*, dobbiamo assicurarci che i suoi due parametri (data e dbList) siano correttamente trasmessi sulla rete e interpretati dal server. Alla fine, il server riceverà solo una serie di byte che rappresentano il messaggio originale inviato dal client. Tuttavia, nessuna informazione aggiuntiva sul significato di quei byte è tipicamente inclusa nel messaggio, per non parlare del fatto che dovremmo affrontare di nuovo lo stesso problema: come dovrebbe il server riconoscere la meta-informazione in quanto tale?

Oltre a questo problema di interpretazione, dobbiamo anche tenere conto della possibilità che il posizionamento dei byte in memoria possa variare tra le architetture delle macchine.

In particolare, dobbiamo tenere conto del fatto che alcune macchine, come l'Intel Pentium, numerano i loro byte da destra a sinistra, mentre molti altri, come i vecchi processori ARM, li numerano nella direzione opposta (ARM ora supporta entrambi). Il formato Intel è indicato come little endian, mentre il formato ARM (più vecchio) è indicato come big endian. L'ordinamento dei byte è anche essenziale per il networking, come possiamo osservare quando le macchine trasmettono (e quindi ricevono) bit e byte in una varietà di ordini. Tuttavia, il big endian viene in genere utilizzato quando si trasferiscono byte su una rete.

La soluzione a questo problema è trasformare i dati da trasmettere in un formato indipendente dalla macchina e dalla rete, oltre a garantire che entrambe le parti comunicanti anticipino la trasmissione dello stesso tipo di dati del messaggio.

Tipicamente, quest'ultimo può essere risolto a livello di linguaggio di programmazione. L'utilizzo di routine dipendenti dalla macchina che trasformano i dati in entrata ed in uscita in formati indipendenti dalla macchina e dalla rete è una soluzione a questo problema.

Marshaling e unmarshalling sono componenti essenziali delle chiamate di procedura remota a causa della loro attenzione alla conversione in un formato neutro.

Ora ci troviamo di fronte a un problema difficile: come vengono passati i puntatori e i riferimenti in generale? La risposta corretta è: con estrema difficoltà. Un puntatore è significativo solo all'interno dello spazio degli indirizzi del processo attualmente in esecuzione. Tornando al nostro esempio di accodamento, abbiamo affermato che il secondo parametro, dbList, è implementato come riferimento a un elenco

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

memorizzato nel database. Se tale riferimento è semplicemente un puntatore a una struttura di dati locale nella memoria principale del chiamante, non possiamo semplicemente passarlo al server. Il valore del puntatore trasferito farà probabilmente riferimento a un oggetto completamente diverso.

Vietare semplicemente puntatori e parametri di riferimento è una possibile soluzione. Tuttavia, a causa del loro significato e la loro importanza, questa soluzione è altamente indesiderabile. Anzi, in realtà, si rivela spesso inutile. Innanzitutto, i parametri di riferimento vengono spesso utilizzati con tipi di dati con dimensioni fisse, ad esempio matrici statiche, o con tipi di dati dinamici la cui dimensione può essere facilmente calcolata in fase di esecuzione, ad esempio stringhe o matrici dinamiche. In tali situazioni, possiamo semplicemente copiare l'intera struttura dati a cui si riferisce il parametro, sostituendo efficacemente copy-by-value/restore con copy-by-reference. Sebbene ciò non sia sempre semanticamente identico, è spesso adeguato. Quando il client stub è consapevole che i dati di riferimento verranno solo letti, non è necessario copiarli nuovamente quando la chiamata è stata completata. La copia per valore è quindi sufficiente.

In genere, possono essere supportati anche tipi di dati più complessi, a condizione che siano supportati dal linguaggio di programmazione. Un linguaggio che supporta classi definite dall'utente, ad esempio Python o Java, consente a un sistema di linguaggio di fornire marshalling e unmarshalling completamente automatizzati di tali tipi di dati. In particolare, il (un)marshalling automatico potrebbe non essere disponibile o addirittura desiderabile quando si ha a che fare con strutture di dati dinamici estremamente grandi, nidificate o altrimenti complesse.

Come discusso finora, il problema con i puntatori e i riferimenti è che hanno senso solo localmente: si riferiscono a posizioni di memoria che sono significative solo per il processo di chiamata. L'utilizzo di riferimenti globali, ovvero riferimenti significativi sia per il processo chiamante che per quello chiamato, può ridurre i problemi. Se il client e il server hanno accesso allo stesso file system, ad esempio, potrebbe essere sufficiente passare un handle del file anziché un puntatore. Un'osservazione importante è che entrambi i processi devono capire esattamente cosa fare quando viene passato un riferimento globale. In altre parole, se consideriamo un riferimento globale con un tipo di dati associato, i processi chiamanti e chiamati dovrebbero avere una comprensione identica delle possibili operazioni.

Inoltre, entrambi i processi devono concordare esattamente cosa fare quando viene passato un handle del file. Ancora una volta, questi sono in genere problemi che possono essere risolti con il supporto di un linguaggio di programmazione appropriato.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

3. Supporto ad applicazioni basate su RPC

Da quanto abbiamo discusso finora, è evidente che nascondere una chiamata di procedura remota richiede che il chiamante e il destinatario della chiamata concordino il formato dei messaggi che si scambiano e seguano gli stessi passaggi quando si passano strutture di dati complesse. In altre parole, entrambi i lati in una chiamata RPC devono aderire allo stesso protocollo affinché la stessa RPC funzioni correttamente.

Esistono almeno due modi per supportare lo sviluppo di applicazioni basate su RPC. Il primo è quello di consentire allo sviluppatore di specificare con precisione ciò che deve essere chiamato in remoto, da cui possono essere generati stub lato client e lato server. La seconda strategia consiste nell'incorporare la chiamata di procedura remota in un ambiente di linguaggio di programmazione.

Si consideri la funzione *someFunction*:

void someFunction(char x; float y; int z[5])

Un carattere, un numero float e una matrice di cinque numeri interi sono i suoi tre parametri. Supponendo che una parola sia composta da quattro byte, il protocollo RPC potrebbe richiedere la trasmissione di un carattere nel byte più a destra di una parola (lasciando vuoti i tre byte seguenti), di un float come parola intera e di una matrice come gruppo di parole uguale alla lunghezza della matrice, preceduta da una parola che indica la lunghezza, come illustrato nella Figura 3).

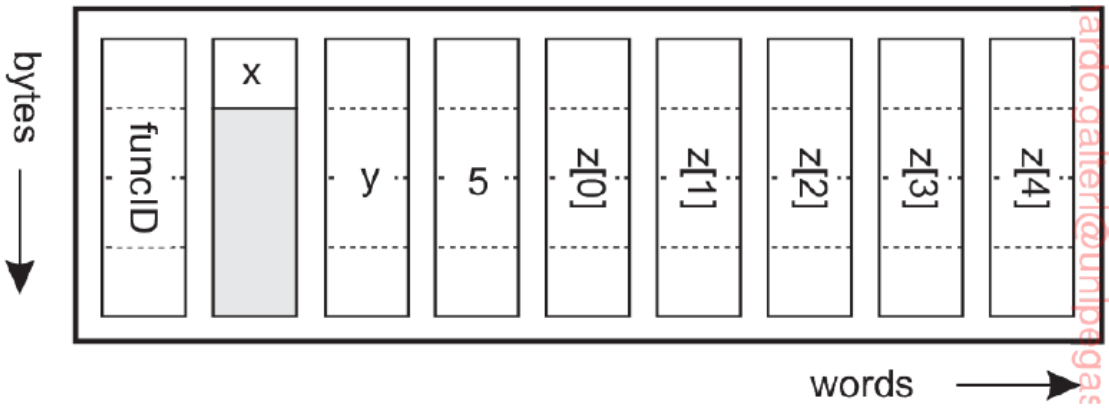


Figura 3 - Courtesy of van Steen, Maarten & Tanenbaum, Andrew. (2017). Distributed Systems, Third edition

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Pertanto, date queste regole, il client stub per *someFunction* è consapevole che deve utilizzare il formato specificato in Figura 3 e lo stub del server è consapevole che i messaggi in arrivo per someFunction avranno il formato specificato in Figura 3.

La definizione del formato del messaggio è una componente essenziale di un protocollo RPC, ma non è sufficiente. Il client e il server devono anche concordare sulla rappresentazione di strutture dati semplici, come numeri interi, caratteri, booleani, ecc. Il protocollo potrebbe specificare, ad esempio, che gli interi sono rappresentati nel complemento di due, i caratteri in Unicode a 16 bit e float nel formato standard IEEE #754, con tutto memorizzato in little endian. Con queste informazioni aggiuntive, i messaggi possono essere interpretati senza ambiguità.

L'unica cosa che resta da fare è che il chiamante e il destinatario della chiamata concordino l'effettivo scambio di messaggi, poiché le regole di codifica sono state determinate nei minimi dettagli. Ad esempio, è possibile scegliere un servizio di trasporto orientato alla connessione, ad esempio TCP/IP. Utilizzare un servizio di datagrammi inaffidabile e consentire al client e al server di implementare uno schema di controllo degli errori come parte del protocollo RPC come alternativa. In pratica, ci sono più varianti, ed è responsabilità dello sviluppatore indicare il servizio di comunicazione sottostante preferito.

I client stub ed i server stub devono essere implementati una volta che il protocollo RPC è stato completamente definito. Fortunatamente, gli stub per lo stesso protocollo ma procedure diverse in genere differiscono solo nella loro interfaccia applicativa. Un'interfaccia è una raccolta di procedure implementate dal server che possono essere chiamate da un client. In genere, un'interfaccia è disponibile nello stesso linguaggio di programmazione in cui è scritto il client o il server (anche se questo è strettamente parlando, non necessario). Le interfacce sono spesso specificate per mezzo di un linguaggio di definizione dell'interfaccia al fine di semplificare le cose (IDL). L'interfaccia specificata in tale IDL viene quindi compilata in un client stub, un server stub e le interfacce appropriate.

In pratica, l'utilizzo di un linguaggio di definizione dell'interfaccia semplifica notevolmente le applicazioni client-server basate su RPC. Poiché è semplice generare stub client e server nella loro interezza, tutti i sistemi middleware basati su RPC forniscono un IDL per facilitare lo sviluppo di applicazioni. In alcuni casi, l'IDL è obbligatorio.

La metodologia descritta finora è in gran parte indipendente da uno specifico linguaggio di programmazione. In alternativa, possiamo incorporare la chiamata di procedura remota direttamente in un linguaggio di programmazione. Il vantaggio principale è che lo sviluppo di applicazioni è spesso semplificato. Inoltre, raggiungere un elevato livello di trasparenza dell'accesso è spesso più semplice perché molti problemi associati al passaggio dei parametri possono essere completamente evitati.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Java, in cui un RPC viene definito Remote Method Invocation (RMI), è un caso noto in cui la chiamata di procedura remota è completamente incorporata. Un client in esecuzione sulla propria macchina virtuale (Java) può essenzialmente richiamare un metodo di un oggetto gestito da un'altra macchina virtuale. Leggendo il codice sorgente di un'applicazione, può essere difficile o impossibile determinare se una chiamata al metodo è a un oggetto locale o remoto.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- M. van Steen and A.S. Tanenbaum, Distributed Systems, 3rd ed., distributed-systems.net, 2017.
- Birrell A. and Nelson B. Implementing Remote Procedure Calls. ACM Transactions on Computer Systems, Feb. 1984.

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).